POLITECHNIKA WROCŁAWSKA
WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

PROJEKT Z BAZ DANYCH

System obsługi zarządzania siłowniami „Fitness-hub”

AUTORY:

Uladzislau Budziankou
Indeks: 273039

Matsvei Samakar
Indeks: 274002

PROWADZĄCY ZAJĘCIA:

Dr inż. Robert Wójcik, K30W04D03

OCENA PRACY:

Wrocław, 2025

Spis treści

1 Wstęp **6**
1.1 Cel projektu . . . 6
1.2 Zakres projektu . . . 6

2 Analiza wymagań **8**
2.1 Opis działania i schemat logiczny systemu . . . 8
2.2 Wymagania funkcjonalne . . . 9
2.2.1 Diagram przypadków użycia . . . 9
2.2.2 Scenariusze wybranych przypadków użycia . . . 9
2.3 Wymagania niefunkcjonalne . . . 11
2.3.1 Wykorzystywane technologie i narzędzia . . . 11
2.3.2 Wymagania dotyczące rozmiaru bazy danych . . . 12
2.3.3 Wymagania dotyczące bezpieczeństwa systemu . . . 12
2.4 Przyjęte założenia projektowe . . . 13

3 Projekt systemu **14**
3.1 Projekt bazy danych . . . 14
3.1.1 Analiza rzeczywistości i uproszczony model konceptualny . . . 14
3.1.2 Model logiczny i normalizacja . . . 15
3.1.3 Model fizyczny i ograniczenia integralności danych . . . 17
3.1.4 Mechanizmy przetwarzania danych . . . 19
3.1.5 Projekt mechanizmów bezpieczeństwa na poziomie bazy danych . . . 19
3.2 Projekt aplikacji użytkownika . . . 19
3.2.1 Architektura aplikacji . . . 20
3.2.2 Interfejs graficzny i struktura menu . . . 21
3.2.3 Metoda podłączania do bazy danych – integracja z bazą danych . . . 25

4 Implementacja systemu **27**
4.1 Realizacja bazy danych . . . 27
4.2 Tworzenie tabel i definiowanie ograniczeń . . . 28
4.3 Implementacja mechanizmów przetwarzania danych . . . 28
4.4 Implementacja uprawnień i innych zabezpieczeń . . . 29
4.5 Realizacja elementów aplikacji . . . 29
4.6 Obsługa menu . . . 29
4.7 Walidacja danych oraz komunikacja z użytkownikiem . . . 29
4.8 Tworzenie serwera aplikacji . . . 29
4.8.1 Implementacja wybranych funkcjonalności systemu . . . 31

5 Testowanie systemu **33**
5.1 Instalacja i konfigurowanie systemu . . . 33
5.2 Testowanie opracowanych funkcji systemu . . . 33
5.2.1 Testowanie funkcji . . . 33
5.2.2 Testy walidacji danych . . . 37
5.2.3 Testy manualne . . . 37
5.3 Wnioski z testów . . . 38

6 Podsumowanie **39**

7 Literatura i źródła **40**

Spis rysunków

1 Model logiczny systemu 8
2 Diagram przypadków użycia 9
3 Diagram modelu konceptualnego 14
4 Diagram modelu logicznego 15
5 Diagram modelu logicznego 17
6 Strona główna aplikacji 22
7 Mapa siłowni aplikacji 23
8 Lista siłowni aplikacji 23
9 Szczegóły siłowni i dodawanie recenzji aplikacji 24
10 Formularz dodawania siłowniaplikacji 25
11 Kod połączenia z bazą danych 30
12 Kod obsługi tras 30
13 Kod obsługi użytkowników 31
14 Testy funkcji (1) 34
15 Testy funkcji (2) 35
16 Testy funkcji (3) 36
17 Testy funkcji (4) 36
18 Test walidacji danych 37

Spis tabel

1 Tabela narzędzi użytych do integracji z bazą danych 26
2 Tabela endpointów API systemu WellFitness 28

1 Wstęp

W ostatnich latach coraz więcej osób decyduje się na prowadzenie aktywnego trybu życia, co przekłada się na rosnącą popularność siłowni i klubów fitness. Dzięki rozwojowi technologii użytkownicy mają dziś większe możliwości wyszukiwania usług i porównywania ich w wygodny sposób, za pomocą aplikacji i platform internetowych. Jednak wciąż brakuje uniwersalnego narzędzia, które pozwalałoby w prosty sposób znaleźć siłownię w swojej okolicy, porównać ceny karnetów, zapoznać się z opiniami innych użytkowników oraz dodać własne recenzje.

Aby odpowiedzieć na te potrzeby, powstał projekt „Fitness-hub" – system internetowy, który umożliwia kompleksowe zarządzanie informacjami o siłowniach. Platforma pozwala użytkownikom przeglądać lokalizacje siłowni na mapie lub w formie listy, sprawdzać ceny, oceniać jakość usług oraz dzielić się opiniami. Administratorzy natomiast mają dostęp do zaawansowanych funkcji, takich jak zarządzanie danymi o siłowniach, recenzjami czy użytkownikami, co zapewnia kontrolę nad jakością prezentowanych treści.

Projekt ten ma na celu ułatwienie użytkownikom podejmowania decyzji dotyczących wyboru siłowni, przy jednoczesnym zwiększeniu widoczności i konkurencyjności klubów fitness na rynku. „Fitness-hub" został zaprojektowany z myślą o intuicyjności, przejrzystości oraz wygodzie użytkowania, co czyni go idealnym narzędziem zarówno dla osób aktywnie korzystających z siłowni, jak i dla właścicieli, którzy chcą promować swoje obiekty.

1.1 Cel projektu

Celem projektu jest stworzenie kompleksowego systemu internetowego do zarządzania siłowniami, który umożliwi zarówno administratorom, jak i użytkownikom efektywne zarządzanie danymi oraz korzystanie z usług siłowni. System zapewnia administratorom możliwość dodawania, edytowania i usuwania danych dotyczących siłowni, recenzji oraz użytkowników. Ułatwia również zarządzanie kontami użytkowników poprzez ich edycję, blokowanie i usuwanie, a także moderowanie recenzji w celu utrzymania wysokiej jakości treści. Administratorzy mają pełną kontrolę nad bazą danych, co pozwala na wykonywanie wszelkich operacji związanych z danymi dotyczącymi siłowni, użytkowników i recenzji.

Dla użytkowników system oferuje intuicyjne i wygodne narzędzia, takie jak możliwość dodawania recenzji siłowni, oceniania ich usług oraz przeglądania szczegółowych informacji, takich jak lokalizacja, godziny otwarcia czy dane kontaktowe. Użytkownicy mogą także korzystać z interaktywnej mapy, która pozwala na szybkie znajdowanie siłowni w pobliżu, przeglądanie ich na mapie oraz wyznaczanie trasy do wybranego obiektu. System został zaprojektowany z myślą o przejrzystości, dostępności i wygodzie użytkowania, co ma na celu zapewnienie pozytywnych doświadczeń zarówno dla administratorów, jak i użytkowników, przy jednoczesnym zwiększeniu efektywności zarządzania danymi i usługami siłowni.

1.2 Zakres projektu

Projekt zakłada stworzenie i wdrożenie internetowego systemu zarządzania siłowniami o nazwie „Fitness-hub". System ten umożliwia użytkownikom przeglądanie dostępnych siłowni na mapie i w formie listy, sprawdzanie cen karnetów oraz dodawanie ocen i komentarzy. Dodatkowo, zalogowani użytkownicy mają możliwość dodawania, edytowania

oraz usuwania siłowni, a także zarządzania swoimi recenzjami. Administratorzy systemu mogą moderować recenzje, zarządzać kontami użytkowników i danymi siłowni.

System oparty jest na technologii Node.js (backend) i PostgreSQL (baza danych). Interfejs użytkownika został zaprojektowany z użyciem HTML5, CSS3, JavaScript oraz frameworka Bootstrap. Projekt obejmuje pełną implementację warstwy frontendowej i backendowej, ale nie zawiera dedykowanej aplikacji mobilnej ani integracji z zewnętrznymi systemami.

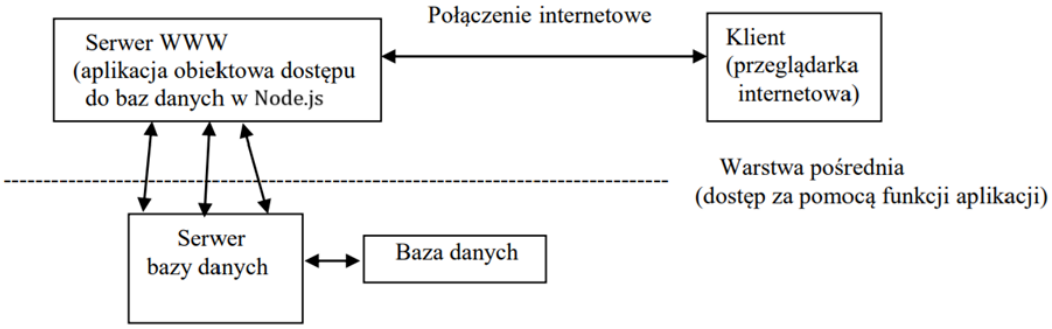
Zakres projektu obejmuje:

- Zaprojektowanie i implementację bazy danych oraz logiki serwera.
- Stworzenie responsywnego interfejsu użytkownika.
- Zapewnienie funkcjonalności takich jak rejestracja, logowanie, zarządzanie siłowniami i recenzjami.
- Wdrożenie podstawowych mechanizmów bezpieczeństwa, w tym szyfrowania haseł i uwierzytelniania użytkowników.

2 Analiza wymagań

Główne założenia projektu obejmują modularną architekturę systemu, umożliwiającą jego przyszłą rozbudowę, integrację z zewnętrznymi usługami oraz zapewnienie skalowalności dla rosnącej liczby użytkowników. W ramach projektu opisano organizację danych, kluczowe relacje między encjami oraz sposoby przetwarzania informacji.

2.1 Opis działania i schemat logiczny systemu



Rysunek 1: Model logiczny systemu

System WellFitness został zaprojektowany jako aplikacja internetowa umożliwiająca użytkownikom wyszukiwanie siłowni, przeglądanie szczegółowych informacji o obiektach oraz dodawanie recenzji. Działanie systemu opiera się na architekturze klient-serwer, w której poszczególne warstwy odpowiadają za różne aspekty obsługi aplikacji.

Warstwy systemu i przepływ danych:

Warstwa klienta
Użytkownik korzysta z przeglądarki internetowej (klient), aby uzyskać dostęp do systemu. Przesyła żądania dotyczące przeglądania siłowni, dodawania recenzji czy zarządzania swoim kontem. Wszystkie operacje wykonywane przez użytkownika są przekazywane do serwera aplikacji.

Warstwa pośrednia (Serwer WWW)
Serwer aplikacyjny, napisany w Node.js, pełni rolę warstwy logicznej systemu. Odpowiada za obsługę żądań klientów, przetwarzanie danych oraz komunikację z bazą danych. W ramach tej warstwy realizowane są operacje REST API, takie jak pobieranie listy siłowni, dodawanie nowych wpisów czy uwierzytelnianie użytkowników.

Warstwa bazy danych
System korzysta z relacyjnej bazy danych PostgreSQL[1], w której przechowywane są informacje o użytkownikach, siłowniach i recenzjach. Dostęp do danych odbywa się wyłącznie poprzez serwer aplikacyjny, co zapewnia bezpieczeństwo i kontrolę nad operacjami na bazie.

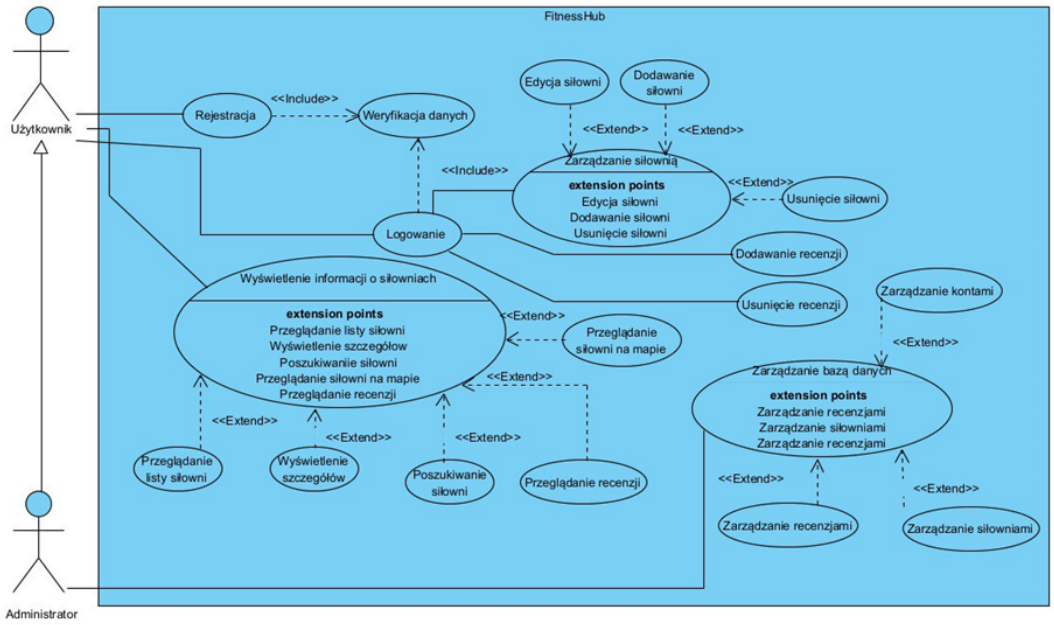
2.2 Wymagania funkcjonalne

W trakcie analizy wymagań została zdefieniowana podstawowa lista wymagań systemu:

1. Użytkownik ma możliwość założenia konta.
2. Użytkownik ma możliwość logowania się na stronie.
3. Użytkownik ma możliwość przeglądania siłowni na mapie.
4. Użytkownik może przeglądać poszczególną informację wybranej siłowni.
5. Użytkownik może przeglądać recenzje innych użytkowników i swoje.
6. System automatycznie usuwa powiązanych recenzji po usunięciu siłowni.

2.2.1 Diagram przypadków użycia

W celu określenia funkcjonalności systemu oraz interakcji pomiędzy użytkownikami a systemem został utworzony diagram przypadków użycia. Jego zadaniem jest zobrazowanie ról (aktorów) oraz operacji, które mogą być przez nich wykonywane, co pozwala na lepsze zrozumienie wymagań systemowych.



Rysunek 2: Diagram przypadków użycia

2.2.2 Scenariusze wybranych przypadków użycia

Poniżej znajdują się przykładowe scenariusze przypadków użycia, przedstawiające interakcje użytkowników z systemem oraz opisujące sposób realizacji poszczególnych funkcjonalności.
Przypadek użycia 1: Rejestracja.
Cel: Pozwala użytkownikowi na założenie konta w systemie.
Warunki wstępne: Brak konta użytkownika w systemie.

Warunki końcowe: Konto użytkownika zostało poprawnie zarejestrowane i zapisane w bazie danych.
Scenariusz:

1. Użytkownik uruchamia proces rejestracji.
2. System wyświetla formularz rejestracji.
3. Użytkownik wypełnia wymagane pola formularza (np. username, e-mail, hasło).
4. Użytkownik zatwierdza formularz.
5. System weryfikuje poprawność wprowadzonych danych.
6. Jeśli dane są poprawne, system zapisuje konto użytkownika w bazie danych, jeżeli są błędne, to informuje o błędzie.
7. System informuje użytkownika o pomyślnej rejestracji albo prosi o weryfikację danych.

Przypadek użycia 2: Weryfikacja danych
Cel: Weryfikacja poprawności danych wprowadzonych przez użytkownika podczas rejestracji
Warunki wstępne: Użytkownik wybrał opcję rejestracji i wprowadził wymagane dane
Warunki końcowe: Dane są poprawne i mogą być użyte do utworzenia konta; użytkownik jest gotowy do zakończenia rejestracji
Scenariusz:

1. Użytkownik wprowadza dane podczas rejestracji.
2. System sprawdza, czy wszystkie wymagane pola zostały wypełnione i czy dane są zgodne z wymaganiami.
3. Jeśli dane są poprawne, system kontynuuje proces rejestracji.
4. W przeciwnym wypadku prosi o poprawne wprowadzenie danych.

Przypadek użycia 3: Logowanie.
Cel: Umożliwienie zarejestrowanemu użytkownikowi zalogowania się do systemu FitnessHub.
Warunki wstępne: Konto użytkownika istnieje w systemie.
Warunki końcowe: Użytkownik jest zalogowany i ma dostęp do funkcji systemu. Scenariusz:

1. Użytkownik wybiera opcję „Logowanie”.
2. System wyświetla formularz logowania.
3. Użytkownik wprowadza dane logowania (username, e-mail, hasło).
4. System weryfikuje poprawność danych logowania.
5. Jeśli dane są poprawne, system loguje użytkownika i wyświetla stronę główną.

Przypadek użycia 4: Wyświetlenie informacji o siłowniach.
Cel: Umożliwienie użytkownikowi przeglądania dostępnych siłowni w systemie.
Warunki wstępne: Użytkownik uruchamia stronę internetową.
Warunki końcowe: Strona główna z mapą.
Scenariusz:

1. Użytkownik uruchamia stronę internetową
2. System wita użytkownika
3. Użytkownik wybiera opcję „Zobacz siłowni”
4. System wyświetla informację

2.3 Wymagania niefunkcjonalne

Została zdefieniowana następna lista wymagań niefunkcjonalnych:

1. Tylko zalogowany użytkownik ma możliwość utworzyć swoją siłownię (staje się autorem).
2. Tylko zalogowany użytkownik może dodać nową recenzję siłowni.
3. Tylko zalogowany użytkownik ma możliwość zarządzania swoją siłownią oraz ją danymi.
4. Tylko zalogowany użytkownik ma możliwość usuwania jakiejkolwiek recenzji dla swojej siłowni.
5. Tylko zalogowany użytkownik może edytować i usuwać swoje recenzje.
6. Administrator ma możliwość edytowania oraz usuwania istniejących siłowni z bazy danych oraz zarządzaniem kontami i recenzjami użytkowników.
7. System weryfikuje wprowadzone dane podczas rejestracji, logowania, utworzenia siłowni oraz recenzji.
8. System zapewnia bezpieczne przechowywanie haseł użytkowników.

2.3.1 Wykorzystywane technologie i narzędzia

W celu zapewnienia stabilności, wydajności oraz łatwości rozwoju systemu wykorzystano nowoczesne technologie backendowe, bazodanowe oraz frontendowe. Wybór tych narzędzi pozwala na efektywne zarządzanie danymi oraz intuicyjną obsługę interfejsu użytkownika.

Backend:

- **Node.js[2]** – Nowoczesna, asynchroniczna platforma wykonawcza dla JavaScript, umożliwiająca tworzenie skalowalnych aplikacji serwerowych. Dzięki obsłudze architektury event-driven zapewnia szybkie i efektywne przetwarzanie żądań użytkowników.

- **Express.js[3]** – Lekki i elastyczny framework webowy dla Node.js, usprawniający zarządzanie trasami (routing), obsługę żądań HTTP oraz integrację z middleware, co przyspiesza implementację serwera aplikacji.

Baza danych:

- **PostgreSQL[1]** – Stabilny i wydajny system zarządzania relacyjną bazą danych. Oferuje wsparcie dla transakcji, normalizacji danych oraz skomplikowanych zapytań SQL, co pozwala na efektywne przechowywanie i przetwarzanie informacji o użytkownikach, recenzjach oraz siłowniach.

Frontend:

- **HTML5, CSS3, JavaScript[8]** – Podstawowe technologie do budowy interaktywnego interfejsu użytkownika. Zapewniają responsywność oraz nowoczesny wygląd aplikacji na różnych urządzeniach.

2.3.2 Wymagania dotyczące rozmiaru bazy danych

System bazodanowy został zaprojektowany z uwzględnieniem przyszłego skalowania, ale jego początkowa konfiguracja powinna spełniać następujące wymagania:

- Początkowy rozmiar bazy danych nie powinien przekraczać 100 MB, uwzględniając podstawowe dane użytkowników, recenzje oraz informacje o siłowniach.
- System powinien obsłużyć rozbudowę bazy danych do 10 GB bez znaczącego spadku wydajności.
- Każda recenzja użytkownika wraz z powiązanymi danymi (np. zdjęcia) może zajmować maksymalnie 500 KB.
- Wydajność zapytań powinna pozostać stabilna przy wzroście liczby rekordów do 1 miliona.

2.3.3 Wymagania dotyczące bezpieczeństwa systemu

Bezpieczeństwo systemu jest kluczowym aspektem jego funkcjonowania, dlatego zastosowane mechanizmy mają na celu ochronę danych użytkowników oraz zapewnienie integralności systemu.

Wymagania dotyczące bezpieczeństwa obejmują:

Autoryzacja i uwierzytelnianie:

- Użytkownicy muszą logować się do systemu za pomocą unikalnych poświadczeń.

Kontrola dostępu:

- Dostęp do określonych funkcji systemu jest ograniczony na podstawie ról użytkowników (np. administrator, użytkownik standardowy).
- Administratorzy posiadają pełne uprawnienia do zarządzania danymi, podczas gdy użytkownicy mogą edytować jedynie swoje dane.

Bezpieczeństwo transmisji danych:

- Komunikacja między użytkownikami a serwerem odbywa się przy użyciu **protokółu HTTPS** w celu szyfrowania danych.

Bezpieczeństwo bazy danych:

- Regularne **tworzenie kopii zapasowych** i ich przechowywanie w bezpiecznym miejscu.
- Ograniczenie dostępu do bazy danych tylko dla autoryzowanych aplikacji i użytkowników.
- Zastosowanie mechanizmów szyfrowania wrażliwych danych przechowywanych w bazie.

Monitorowanie i audyt:

- Logowanie operacji użytkowników w celu wykrywania potencjalnych zagrożeń.
- Regularne przeglądy zabezpieczeń oraz aktualizacje oprogramowania w celu eliminacji podatności.

2.4 Przyjęte założenia projektowe

Projekt systemu bazodanowego wspomagającego obsługę sklepu internetowego został opracowany na podstawie następujących założeń:

Skalowalność – system powinien być przygotowany na obsługę rosnącej liczby użytkowników i danych, z możliwością późniejszej rozbudowy infrastruktury.

Bezpieczeństwo – zastosowane mechanizmy uwierzytelniania, autoryzacji oraz szyfrowania danych zapewniają ochronę przed nieautoryzowanym dostępem i atakami.

Wydajność – system powinien umożliwiać szybkie operacje na bazie danych, nawet przy dużej ilości rekordów, dzięki optymalizacji indeksów oraz odpowiedniej architekturze aplikacji.

Responsywność interfejsu – aplikacja powinna być dostępna na różnych urządzeniach, dzięki wykorzystaniu technologii frontendowych umożliwiających dynamiczne dostosowanie UI.

Modularność – podział systemu na moduły backendowe, bazodanowe i frontendowe pozwala na łatwiejszą konserwację oraz przyszłą rozbudowę funkcjonalności.

3 Projekt systemu

W tym rozdziale przedstawiono projekt systemu obsługi platformy WellFitness, koncentrując się na kluczowych aspektach związanych z bazą danych, jej projektowaniem oraz architekturą aplikacji. System WellFitness został zaprojektowany w sposób zapewniający wysoką wydajność, bezpieczeństwo oraz intuicyjną obsługę dla użytkowników końcowych, w tym klientów oraz administratorów.

Główne założenia projektu obejmują modularną architekturę systemu, umożliwiającą jego przyszłą rozbudowę, integrację z zewnętrznymi usługami oraz zapewnienie skalowalności dla rosnącej liczby użytkowników. W ramach projektu opisano organizację danych, kluczowe relacje między encjami oraz sposoby przetwarzania informacji.

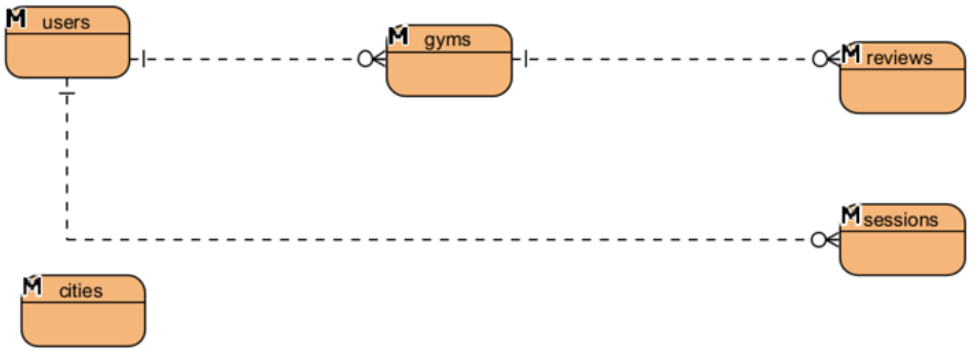
3.1 Projekt bazy danych

W ramach projektowania bazy danych przeprowadzono analizę wymagań systemowych, identyfikując główne encje oraz ich relacje. Na podstawie tych informacji opracowano model konceptualny, który stanowi punkt wyjścia do dalszych etapów projektowania. Struktura bazy danych została zaprojektowana w sposób umożliwiający efektywne zarządzanie danymi dotyczącymi użytkowników, planów treningowych, historii aktywności oraz rezerwacji zajęć w klubach fitness.

Model bazy danych uwzględnia mechanizmy zapewniające integralność danych, ich bezpieczeństwo oraz optymalizację pod kątem wydajności systemu. W kolejnych sekcjach przedstawiono szczegółowy opis struktury bazy, diagramy relacyjne oraz zastosowane techniki indeksowania i optymalizacji zapytań.

3.1.1 Analiza rzeczywistości i uproszczony model konceptualny

System WellFitness został zaprojektowany jako platforma umożliwiająca użytkownikom wyszukiwanie siłowni, rezerwację sesji treningowych oraz wystawianie recenzji. Kluczowym elementem analizy rzeczywistości było zidentyfikowanie głównych encji i ich relacji, co pozwoliło na stworzenie uproszczonego modelu konceptualnego przedstawionego na diagramie.



Rysunek 3: Diagram modelu konceptualnego

Główne encje i ich zależności:

Users (Użytkownicy) – reprezentują osoby korzystające z systemu. Każdy użytkownik może rezerwować sesje treningowe, wystawiać recenzje oraz być powiązany z miastem zamieszkania.

Gyms (Siłownie) – obiekty dostępne w systemie, które mogą być oceniane przez użytkowników. Każda siłownia ma swoją lokalizację oraz powiązanie z miastem.

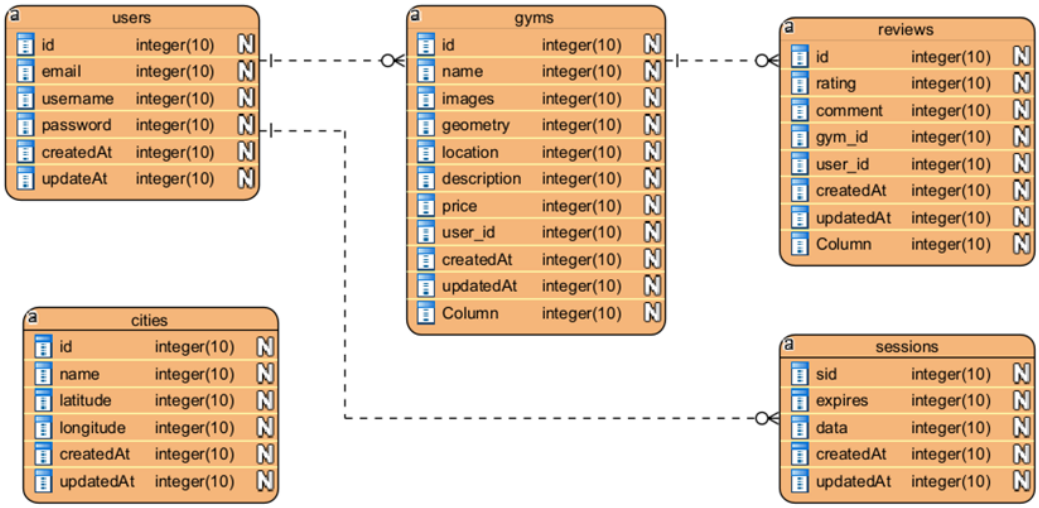
Cities (Miasta) – reprezentują lokalizację siłowni oraz użytkowników, co ułatwia filtrowanie obiektów według lokalizacji.

Sessions (Sesje treningowe) – określają zaplanowane treningi, które użytkownicy mogą rezerwować. Każda sesja jest powiązana z konkretną siłownią.

Reviews (Recenzje) – pozwalają użytkownikom oceniać i komentować siłownie, co wpływa na reputację obiektów w systemie.

3.1.2 Model logiczny i normalizacja

Model logiczny



Rysunek 4: Diagram modelu logicznego

Model logiczny systemu WellFitness został zaprojektowany w sposób zapewniający spójność danych oraz efektywność operacji. Zawiera on kluczowe encje, ich atrybuty oraz relacje pomiędzy nimi. Struktura tabel odzwierciedla rzeczywiste powiązania między użytkownikami, siłowniami, recenzjami, sesjami oraz miastami. W modelu logicznym każda tabela posiada unikalny klucz główny (primary key), a relacje między tabelami są realizowane za pomocą kluczy obcych (foreign keys). Przykładowo:

- Tabela users zawiera dane użytkowników, a jej klucz główny id jest powiązany z recenzjami `reviews.user_id` oraz siłowniami `gyms.user_id`, jeśli użytkownik jest właścicielem siłowni.
- Tabela gyms przechowuje informacje o siłowniach, a jej id jest używane jako klucz obcy w tabelach reviews oraz sessions.
- Tabela cities łączy się z użytkownikami oraz siłowniami, umożliwiając filtrowanie obiektów według lokalizacji.
- Każda recenzja (reviews) oraz sesja treningowa (sessions) są powiązane z użytkownikiem i siłownią, co umożliwia zachowanie integralności danych.

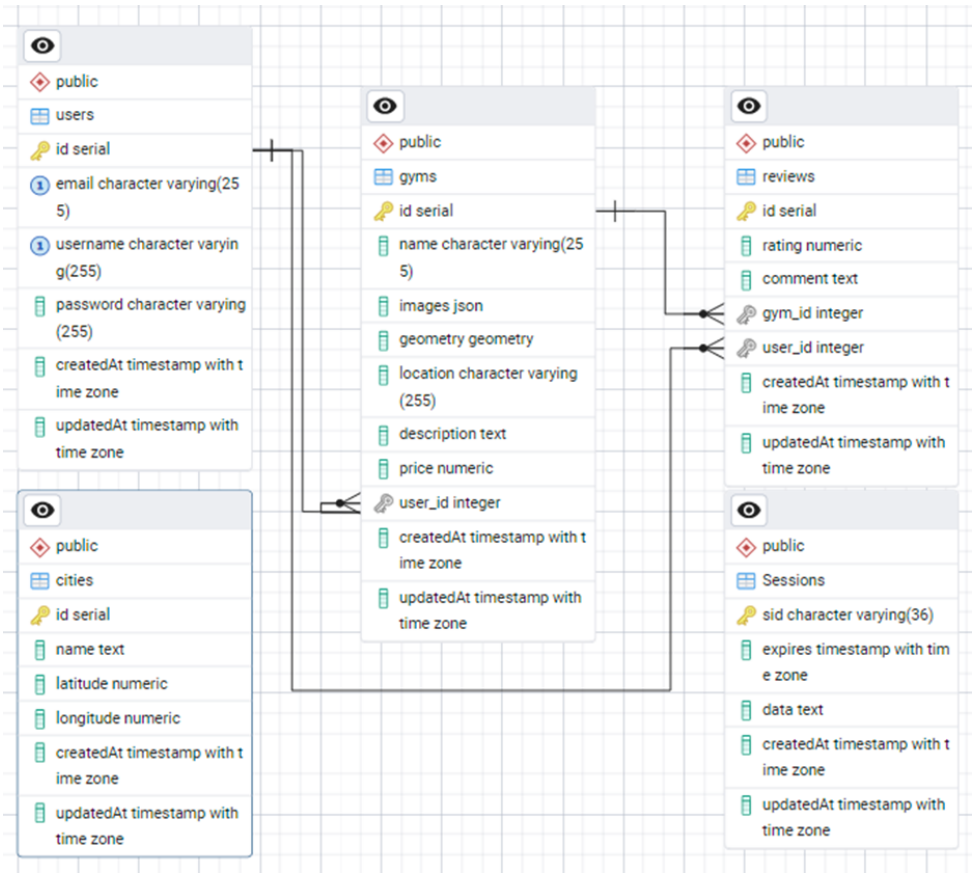
Normalizacja

W celu optymalizacji struktury bazy danych oraz eliminacji redundancji zastosowano proces normalizacji do trzeciej postaci normalnej (3NF), zapewniając:

1. 1NF (Pierwsza postać normalna)
   - Każda kolumna zawiera tylko jedną wartość (atomowość danych).
   - Wiersze są jednoznacznie identyfikowane przez klucz główny. 2NF (Druga postać normalna)
2. 2NF (Druga postać normalna)
   - Wszystkie atrybuty są zależne od całego klucza głównego.
   - Przeniesiono zależności częściowe do oddzielnych tabel (np. informacje o miastach do tabeli cities).
3. 3NF (Trzecia postać normalna)
   - Usunięto zależności przechodnie, dzięki czemu każda tabela zawiera tylko dane bezpośrednio związane z jej kluczem głównym.

   Dzięki tym zabiegom model logiczny bazy danych jest zoptymalizowany pod kątem wydajności, przejrzystości oraz możliwości łatwej rozbudowy w przyszłości.

3.1.3 Model fizyczny i ograniczenia integralności danych



Rysunek 5: Diagram modelu logicznego

Model fizyczny bazy danych systemu WellFitness został zaprojektowany z uwzględnieniem wydajności, bezpieczeństwa oraz spójności danych. Struktura tabel oraz relacje między nimi odzwierciedlają rzeczywiste procesy zachodzące w systemie, zapewniając integralność danych oraz ich logiczną organizację. Poniżej przedstawiono opis tabel oraz ich relacji.

Tabela users

- **Cel:** Przechowuje informacje o użytkownikach systemu.
- **Kolumny:**
  - `id` (serial, klucz główny) – Unikalny identyfikator użytkownika.
  - `email` (character varying) – Adres e-mail użytkownika.
  - `username` (character varying) – Nazwa użytkownika.
  - `password` (character varying) – Hasło użytkownika (prawdopodobnie zaszyfrowane).
  - `createdAt` (timestamp) – Data i czas utworzenia konta.
  - `updatedAt` (timestamp) – Data i czas ostatniej aktualizacji konta.

Tabela cities

- **Cel:** Przechowuje informacje o miastach, w których mogą znajdować się siłownie.

- **Kolumny:**
  - `id` (serial, klucz główny) – Unikalny identyfikator miasta.
  - `name` (text) – Nazwa miasta.
  - `latitude` (numeric) – Szerokość geograficzna miasta.
  - `longitude` (numeric) – Długość geograficzna miasta.
  - `createdAt` (timestamp) – Data i czas utworzenia rekordu miasta.
  - `updatedAt` (timestamp) – Data i czas ostatniej aktualizacji danych miasta.

Tabela gyms

- **Cel:** Przechowuje informacje o siłowniach.
- **Kolumny:**
  - `id` (serial, klucz główny) – Unikalny identyfikator siłowni.
  - `name` (character varying) – Nazwa siłowni.
  - `images` (json) – Obrazy lub grafiki związane z siłownią.
  - `geometry` (geometry) – Dane geograficzne dotyczące lokalizacji siłowni.
  - `location` (character varying) – Opis lokalizacji siłowni.
  - `description` (text) – Opis siłowni.
  - `price` (numeric) – Cena związana z usługami siłowni.
  - `user_id` (integer, klucz obcy) – Odniesienie do użytkownika (właściciela siłowni).
  - `createdAt` (timestamp) – Data i czas utworzenia rekordu siłowni.
  - `updatedAt` (timestamp) – Data i czas ostatniej aktualizacji siłowni.

Tabela reviews

- **Cel:** Przechowuje recenzje siłowni.
- **Kolumny:**
  - `id` (serial, klucz główny) – Unikalny identyfikator recenzji.
  - `rating` (numeric) – Ocena siłowni.
  - `comment` (text) – Komentarz do recenzji.
  - `gym_id` (integer, klucz obcy) – Odniesienie do recenzowanej siłowni.
  - `user_id` (integer, klucz obcy) – Odniesienie do użytkownika, który wystawił recenzję.
  - `createdAt` (timestamp) – Data i czas utworzenia recenzji.
  - `updatedAt` (timestamp) – Data i czas ostatniej aktualizacji recenzji.

Relacje między tabelami

- `users` → `gyms`: Jeden użytkownik może być właścicielem wielu siłowni (relacja 1:N).
- `users` → `reviews`: Jeden użytkownik może napisać wiele recenzji (relacja 1:N).
- `gyms` → `reviews`: Jedna siłownia może mieć wiele recenzji (relacja 1:N).

3.1.4 Mechanizmy przetwarzania danych

W systemie WellFitness zastosowano różne mechanizmy przetwarzania danych w celu zapewnienia ich integralności, wydajności oraz bezpieczeństwa. Przetwarzanie danych obejmuje:

- **Mechanizmy transakcyjne**: Wszystkie operacje na bazie danych są wykonywane w ramach transakcji, co zapewnia spójność danych i umożliwia ich wycofywanie w razie błędów.
- **Walidacja danych**: Przed zapisaniem do bazy, dane użytkowników, recenzji i siłowni przechodzą przez mechanizmy walidacji, co zapobiega błędom i wprowadzaniu nieprawidłowych wartości.
- **Mechanizmy indeksowania**: Wykorzystano indeksy na kluczowych kolumnach (np. `id`, `user_id`, `gym_id`), aby zoptymalizować wydajność zapytań.
- **Bezpieczeństwo dostępu**: System implementuje kontrolę dostępu do danych, zapewniając, że użytkownicy mogą edytować tylko swoje własne recenzje oraz siłownie, których są właścicielami.
- **Optymalizacja zapytań**: Wdrożono strategie optymalizacyjne, takie jak cachowanie zapytań oraz partycjonowanie tabel, aby zwiększyć efektywność działania systemu.

3.1.5 Projekt mechanizmów bezpieczeństwa na poziomie bazy danych

W celu zabezpieczenia danych w bazie **PostgreSQL** systemu WellFitness wdrożono następujące mechanizmy:

- **Role i uprawnienia**:
  - Utworzono różne role dla administratorów, właścicieli siłowni i użytkowników.
- **Kopie zapasowe**:
  - Przechowywanie replik bazy w odrębnej lokalizacji.

Dzięki tym mechanizmom system WellFitness zapewnia bezpieczeństwo przechowywanych danych i chroni je przed nieautoryzowanym dostępem.

3.2 Projekt aplikacji użytkownika

W tym rozdziale przedstawiono projekt aplikacji użytkownika systemu WellFitness. Omówiono kluczowe aspekty interfejsu użytkownika, funkcjonalności aplikacji oraz jej integrację z bazą danych. Aplikacja została zaprojektowana z myślą o intuicyjnej obsłudze, responsywności oraz wysokiej wydajności. System WellFitness pozwala użytkownikom na rejestrację, logowanie, wyszukiwanie siłowni, przeglądanie opinii, rezerwację sesji treningowych oraz ocenianie siłowni. Administratorzy i właściciele siłowni mają dostęp do narzędzi do zarządzania danymi, w tym dodawania nowych siłowni, edytowania opisów oraz przeglądania statystyk.

Aplikacja wykorzystuje nowoczesne technologie frontendowe i backendowe, umożliwiając płynną i bezpieczną obsługę użytkowników. Kluczowym elementem projektu jest

integracja interfejsu użytkownika z bazą danych PostgreSQL, co pozwala na efektywne zarządzanie danymi oraz szybkie wyszukiwanie informacji. W kolejnych sekcjach opisano szczegóły implementacyjne poszczególnych modułów aplikacji.

3.2.1 Architektura aplikacji

Struktura katalogów backendu:

- **config/** – pliki konfiguracyjne, w tym ustawienia bazy danych (`database.js`) oraz inne parametry aplikacji (`config.js`).
- **controllers/** – implementacja logiki biznesowej dla siłowni (`gyms.js`), recenzji (`reviews.js`) i użytkowników (`users.js`).
- **models/** – definicje tabel w bazie danych PostgreSQL, np. `gym.js` (siłownie), `review.js` (recenzje), `user.js` (użytkownicy).
- **routes/** – definicja tras API obsługujących siłownie (`gyms.js`), recenzje (`reviews.js`) i użytkowników (`users.js`).
- **utilities/** – pliki pomocnicze, np. `catchAsync.js` (obsługa błędów asynchronicznych) i `expressError.js` (zarządzanie błędami w Express.js).
- **seeds/** – skrypty do wypełniania bazy danymi testowymi (`seedCities.js`, `gymDescriptions.js`)
- **app.js** – główny plik aplikacji, uruchamiający Express.js oraz konfigurujący middleware i trasy API.
- **middleware.js** – obsługuje funkcje pośredniczące, np. uwierzytelnianie użytkowników i walidację danych.
- **schemas.js** – zawiera schematy walidacyjne dla danych wejściowych.

Sposób działania backendu:

1. Użytkownik wysyła żądanie HTTP do serwera (np. `GET /gyms` – pobranie listy siłowni).
2. Serwer odbiera żądanie, przetwarza je i komunikuje się z bazą PostgreSQL.
3. Dane są pobierane z bazy, a następnie serwer generuje odpowiedź w formacie JSON.
4. Przeglądarka użytkownika otrzymuje dane i wyświetla je w interfejsie graficznym.

Struktura katalogów frontendu:

- **views/** – zawiera szablony EJS odpowiedzialne za generowanie stron:
  - **gyms/** – widoki związane z siłowniami (lista siłowni, szczegóły siłowni).
  - **users/** – widoki związane z użytkownikami (logowanie, rejestracja).
  - **layouts/** – główne układy strony (np. nawigacja, stopka).
  - **partials/** – częściowe komponenty, np. pasek nawigacyjny.
  - **error.ejs** – strona błędów.

  - **home.ejs** – strona główna systemu.

- **public/** – katalog przechowujący statyczne pliki frontendowe:
  - **stylesheets/** – pliki CSS odpowiadające za wygląd strony.
  - **javascripts/** – pliki JavaScript obsługujące interakcję użytkownika.

Sposób działania frontendu:

1. Użytkownik otwiera stronę w przeglądarce i wysyła żądanie do serwera.
2. Backend przetwarza żądanie, pobiera dane z bazy i generuje stronę za pomocą EJS.
3. Serwer zwraca wygenerowaną stronę do przeglądarki użytkownika.
4. Pliki JavaScript obsługują interaktywne elementy, np. formularze i dynamiczne zmiany treści.

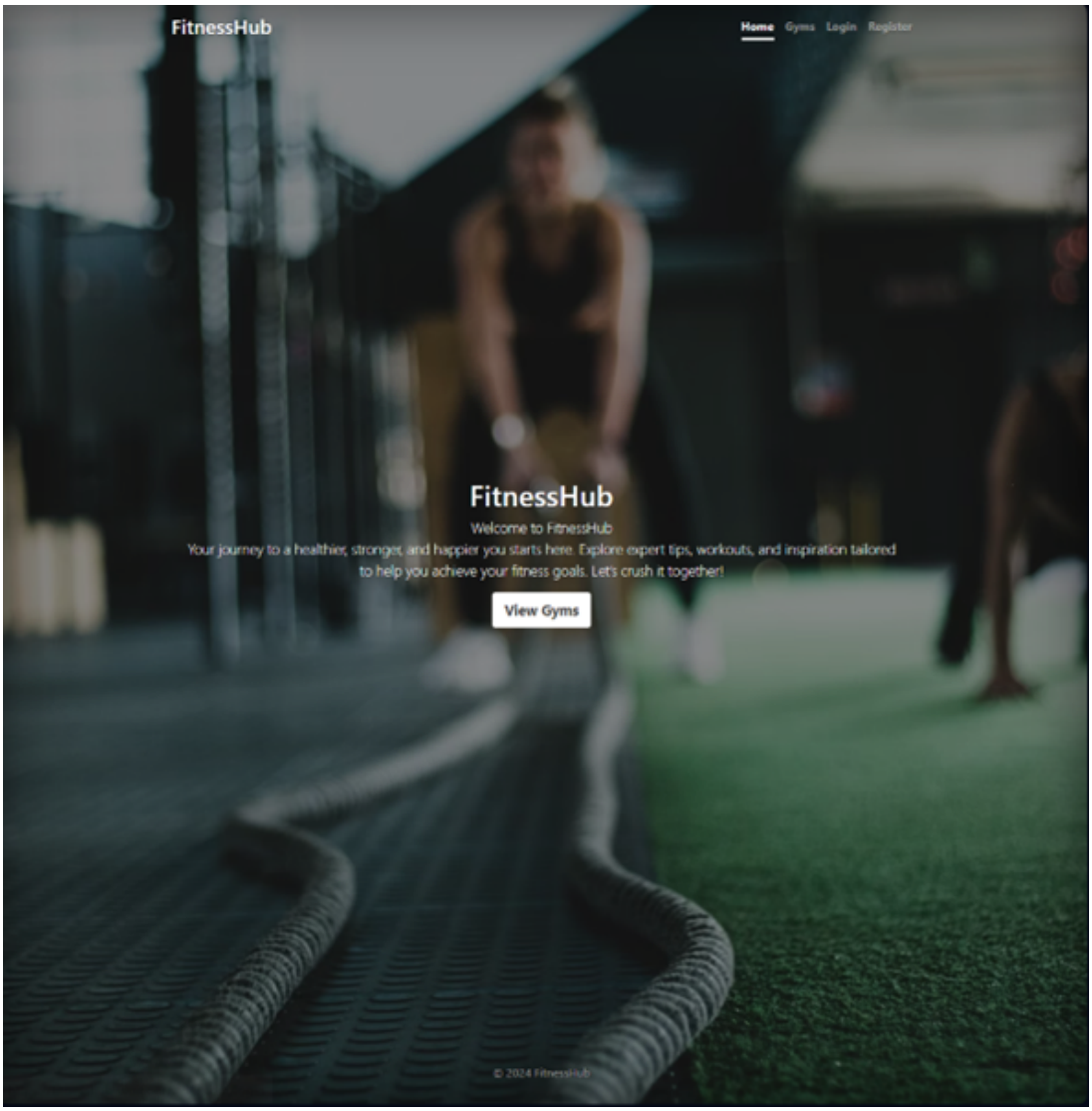
3.2.2 Interfejs graficzny i struktura menu

System WellFitness został zaprojektowany z myślą o intuicyjnej obsłudze, umożliwiając użytkownikom łatwe wyszukiwanie siłowni, przeglądanie szczegółowych informacji oraz dodawanie recenzji. Interfejs graficzny zapewnia przejrzystą nawigację i dostęp do kluczowych funkcji systemu.

Aplikacja składa się z kilku głównych widoków, które pozwalają na wykonywanie podstawowych operacji, takich jak rejestracja i logowanie, przeglądanie siłowni na mapie oraz w formie listy, a także zarządzanie danymi siłowni. Poniżej przedstawiono szczegółowy opis najważniejszych elementów menu oraz funkcjonalności dostępnych w interfejsie użytkownika.

Strona główna

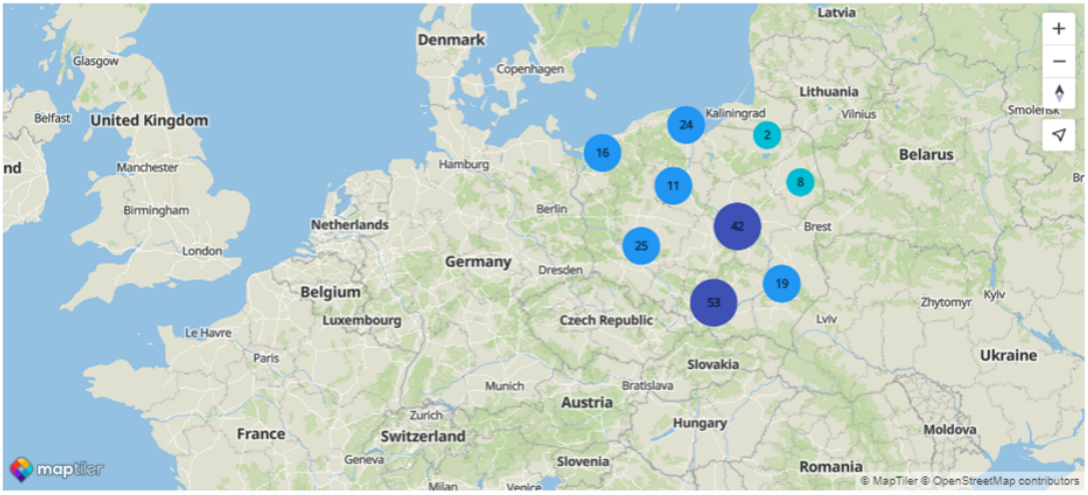
Strona główna aplikacji WellFitness stanowi punkt startowy dla użytkownika. Zawiera tło z motywem siłowni oraz nagłówek powitalny z krótkim opisem funkcjonalności systemu. W centralnej części ekranu umieszczono przycisk „View Gyms”, który przenosi użytkownika do listy dostępnych siłowni. W prawym górnym rogu znajduje się menu nawigacyjne umożliwiające przejście do innych sekcji systemu, takich jak logowanie, rejestracja oraz przeglądanie siłowni.



Rysunek 6: Strona główna aplikacji

Mapa siłowni

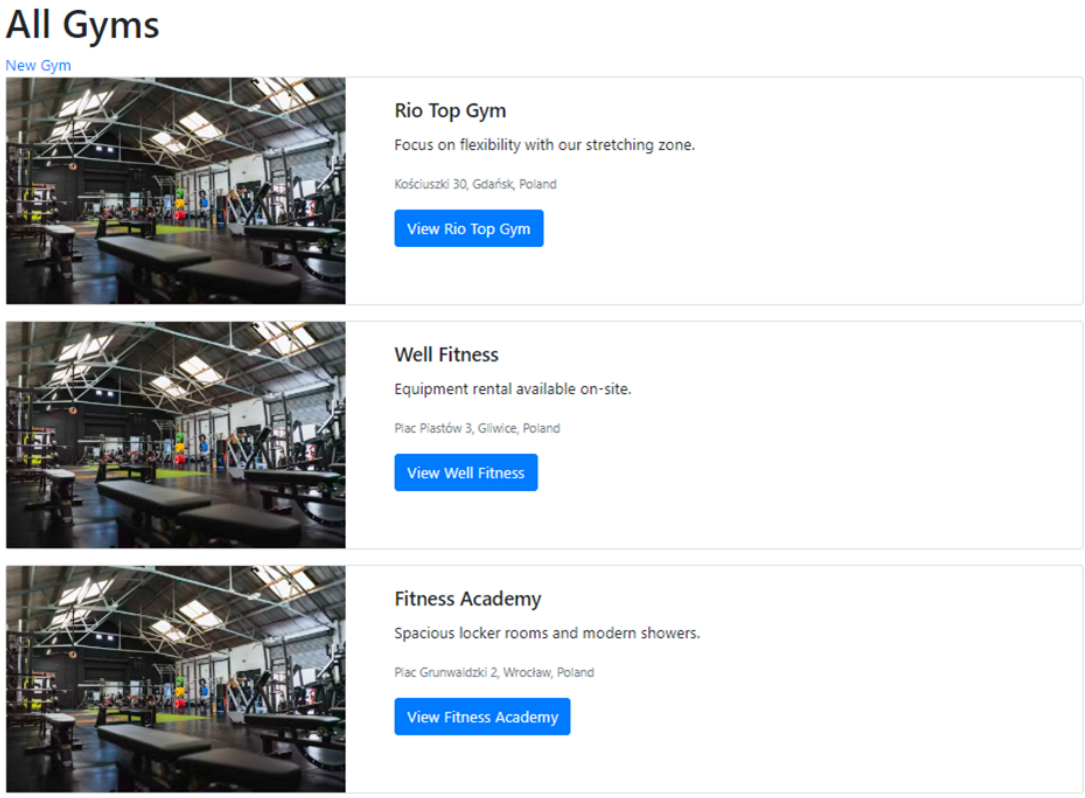
System WellFitness umożliwia wizualizację siłowni na interaktywnej mapie. Każda siłownia jest oznaczona jako klaster, a liczba wyświetlana na znaczniku wskazuje ilość siłowni w danej lokalizacji. Użytkownik może przybliżać i oddalać mapę, a po kliknięciu w znacznik uzyskać dostęp do szczegółowych informacji o siłowniach znajdujących się w wybranym obszarze. Widok mapy pozwala użytkownikom szybko znaleźć siłownię w swojej okolicy.



Rysunek 7: Mapa siłowni aplikacji

Lista siłowni

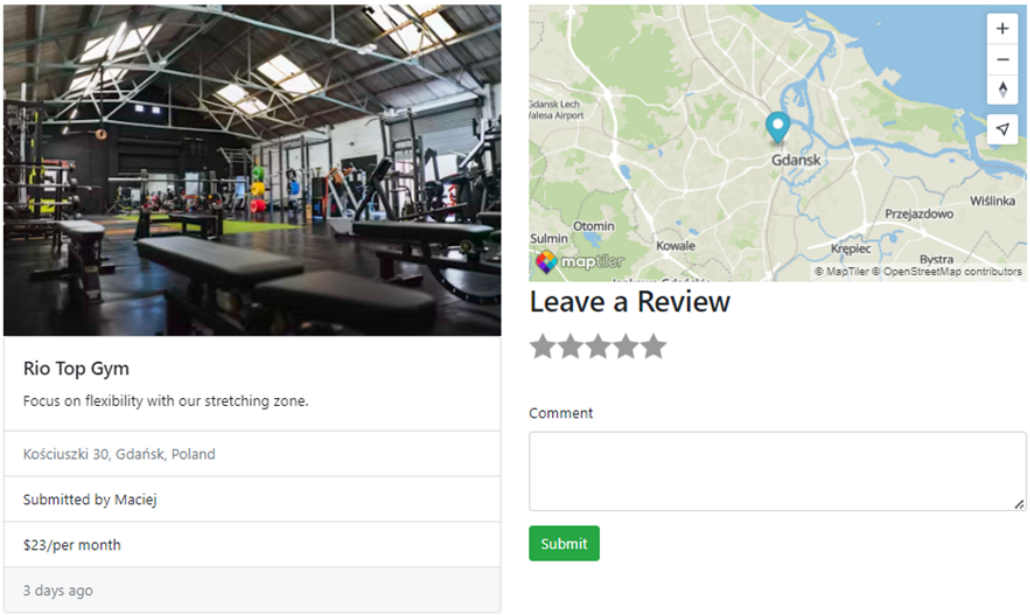
Widok listy siłowni zawiera szczegółowe informacje o dostępnych obiektach sportowych. Każda siłownia jest wyświetlana w postaci karty, która zawiera nazwę, krótki opis, adres oraz zdjęcie. Użytkownik ma możliwość przejścia do szczegółowego widoku siłowni poprzez kliknięcie przycisku „View Gym”. Lista pozwala na szybkie przeglądanie siłowni i wybór odpowiedniego miejsca na podstawie podanych informacji.



Rysunek 8: Lista siłowni aplikacji

Szczegóły siłowni i dodawanie recenzji

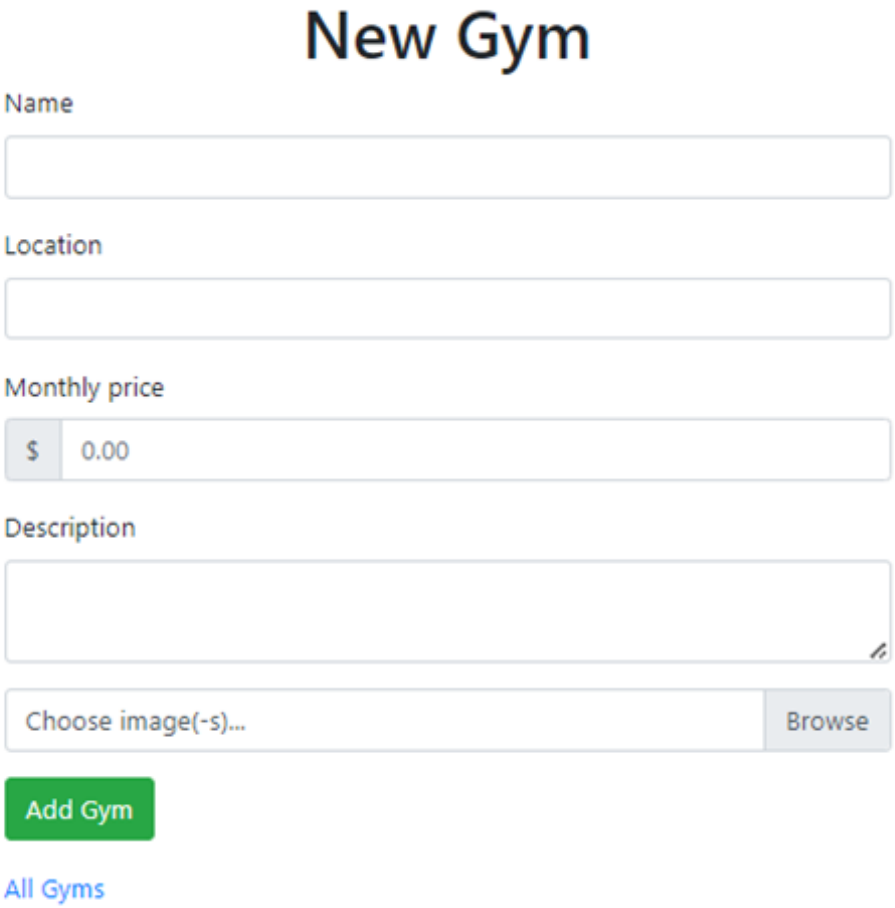
Szczegółowy widok siłowni umożliwia użytkownikowi zapoznanie się z pełnym opisem wybranego obiektu. Strona zawiera nazwę siłowni, jej adres, zdjęcie, koszt karnetu miesięcznego oraz informacje o użytkowniku, który dodał siłownię do systemu. Dodatkowo, użytkownicy mogą pozostawić swoją opinię, wystawiając ocenę w skali gwiazdkowej oraz dodając komentarz. Po dodaniu recenzji jest ona wyświetlana w sekcji opinii dotyczących danej siłowni. Widok zawiera także interaktywną mapę z oznaczoną lokalizacją obiektu.



Rysunek 9: Szczegóły siłowni i dodawanie recenzji aplikacji

Formularz dodawania siłowni

System umożliwia użytkownikom dodawanie nowych siłowni poprzez specjalny formularz. Formularz zawiera pola umożliwiające wpisanie nazwy siłowni, lokalizacji, ceny miesięcznego karnetu oraz opisu. Dodatkowo użytkownik może dodać zdjęcia siłowni, które będą wyświetlane w systemie. Po wypełnieniu formularza i zatwierdzeniu danych przyciskiem „Add Gym” siłownia zostaje zapisana w bazie i pojawia się na liście oraz na mapie dostępnych obiektów.



Rysunek 10: Formularz dodawania siłowniaplikacji

3.2.3 Metoda podłączania do bazy danych – integracja z bazą danych

System WellFitness wykorzystuje bazę danych PostgreSQL hostowaną na serwerze Render.com. Do zarządzania bazą używane jest narzędzie pgAdmin 4, które umożliwia wykonywanie operacji administracyjnych, takich jak tworzenie tabel, edycja danych oraz monitorowanie działania systemu.

Aplikacja backendowa, napisana w Node.js, komunikuje się z bazą danych poprzez bibliotekę Sequelize ORM, co ułatwia operacje CRUD oraz zapewnia abstrakcję nad bezpośrednimi zapytaniami SQL. Połączenie z bazą danych jest konfigurowane w pliku konfiguracyjnym aplikacji, gdzie podawane są dane dostępowe do serwera PostgreSQL na Render.com.

Dzięki zastosowaniu tej metody system zapewnia stabilne i bezpieczne połączenie z bazą danych, umożliwiając efektywne zarządzanie informacjami o użytkownikach, siłowniach i recenzjach.

Komponent	**Narzędzie**	**Funkcja**
Baza danych	PostgreSQL	Przechowywanie danych aplikacji
Zarządzanie bazą	pgAdmin 4	Administracja i monitorowanie bazy
Backend	Node.js	Implementacja logiki aplikacji
Połączenie z bazą	Sequelize ORM	Operacje CRUD na bazie danych
Hosting	Render.com	Hostowanie serwera aplikacji i bazy

Tabela 1: Tabela narzędzi użytych do integracji z bazą danych

4 Implementacja systemu

W tym rozdziale przedstawiono proces implementacji systemu WellFitness, obejmujący realizację bazy danych oraz elementów aplikacji. System został zaprojektowany w sposób modularny, umożliwiający łatwe zarządzanie danymi, bezpieczeństwo oraz skalowalność.

4.1 Realizacja bazy danych

Moduł bazy danych został opracowany przy użyciu PostgreSQL. W ramach implementacji wykonano proces definiowania tabel, ustawiania kluczy głównych i obcych oraz wprowadzenia mechanizmów integralności danych. Poniżej jest przedatawiona tabela użytych endpointów.

Metoda	**Ścieżka (Endpoint)**	**Opis**	**Przykładowa odpowiedź (Response)**
GET	/gyms	Pobiera listę wszystkich siłowni	{ "gyms": [{ "id": 1, "name": "FitZone", "location": "Wrocław", "price": 100 }] }
GET	/gyms/:id	Pobiera szczegóły konkretnej siłowni	{ "id": 1, "name": "FitZone", "location": "Wrocław", "price": 100, "description": "Nowoczesna siłownia" }
POST	/gyms	Tworzy nową siłownię (wymaga autoryzacji)	{ "message": "Siłownia została dodana", "gym": { "id": 2, "name": "BodyFit" } }
PUT	/gyms/:id	Aktualizuje dane siłowni (wymaga autoryzacji)	{ "message": "Siłownia zaktualizowana" }
DELETE	/gyms/:id	Usuwa siłownię (wymaga uprawnień właściciela/admina)	{ "message": "Siłownia została usunięta" }
GET	/reviews/:gymId	Pobiera recenzje dla danej siłowni	{ "reviews": [{ "id": 1, "rating": 5, "comment": "Super miejsce!" }] }
POST	/reviews	Dodaje nową recenzję do siłowni (wymaga autoryzacji)	{ "message": "Recenzja została dodana" }

Metoda	Ścieżka (Endpoint)	Opis	Przykładowa odpowiedź (Response)
DELETE	/reviews/:id	Usuwa recenzję (wymaga uprawnień autora/admina)	{ "message": "Recenzja została usunięta" }
GET	/users/:id	Pobiera dane użytkownika	{ "id": 1, "username": "Janek", "email": "janek@example.com" }
POST	/auth/register	Rejestruje nowego użytkownika	{ "message": "Rejestracja zakończona sukcesem" }
POST	/auth/login	Logowanie użytkownika, zwraca token JWT	{ "token": "eyJhbGciOiJIUzI1NiIsIn..." }
GET	/auth/logout	Wylogowanie użytkownika	{ "message": "Wylogowano pomyślnie" }

Tabela 2: Tabela endpointów API systemu WellFitness

4.2 Tworzenie tabel i definiowanie ograniczeń

Wszystkie tabele zostały utworzone zgodnie z wcześniej zaprojektowanym modelem fizycznym. Zastosowano odpowiednie klucze główne oraz obce, a także ograniczenia integralnościowe, takie jak:

- **NOT NULL** – zapewniające, że kluczowe pola nie pozostaną puste,
- **UNIQUE** – zapobiegające powielaniu kluczowych wartości (np. adresy e-mail użytkowników),
- **CHECK** – walidujące poprawność wprowadzanych danych (np. zakres ocen w recenzjach od 1 do 5),
- **FOREIGN KEY** – zapewniające relacyjność i integralność danych między tabelami.

4.3 Implementacja mechanizmów przetwarzania danych

Aby zapewnić optymalne przetwarzanie danych, w bazie PostgreSQL zaimplementowano:

- Indeksy dla kluczowych kolumn, zwiększające wydajność zapytań.
- Procedury składowane do obsługi złożonych operacji (np. usuwanie powiązanych recenzji po usunięciu siłowni).
- Wyzwalacze (triggers) w celu automatycznego generowania znaczników czasowych.

4.4 Implementacja uprawnień i innych zabezpieczeń

W celu zapewnienia bezpieczeństwa danych:

- Zastosowano system ról i uprawnień w PostgreSQL, ograniczając dostęp do poszczególnych tabel.
- Hasła użytkowników są szyfrowane algorytmem bcrypt przed zapisaniem w bazie.
- Każda transakcja wymagająca zapisu działa w trybie ACID, aby zapewnić spójność danych.

4.5 Realizacja elementów aplikacji

Moduł aplikacji składa się z backendu opartego na Node.js i Express.js oraz interfejsu użytkownika generowanego za pomocą EJS.

4.6 Obsługa menu

System zawiera dynamiczne menu nawigacyjne, dostosowane do poziomu dostępu użytkownika (np. inna nawigacja dla administratora i użytkownika).

4.7 Walidacja danych oraz komunikacja z użytkownikiem

Podczas logowania w systemie WellFitness weryfikowana jest poprawność wprowadzonych danych. Walidacja obejmuje:

- Sprawdzenie poprawnego formatu adresu e-mail (musi zawierać znak „@”).
- Weryfikację, czy konto użytkownika istnieje w bazie danych.
- Obsługę błędnych danych i zwracanie komunikatów walidacyjnych.

System wyświetla komunikaty informacyjne po kluczowych operacjach, takich jak:

- Po dodaniu siłowni – `"Gym successfully added!"`.
- Po dodaniu recenzji – `"Created new review!"` (zielone tło oznacza sukces).
- Po wpisywaniu poprawnej weartości w polu - `"Look's good!"`.
- W przypadku błędu – wyświetlany jest odpowiedni komunikat ostrzegawczy.

4.8 Tworzenie serwera aplikacji

Wdrożenie systemu WellFitness opiera się na platformie Render.com, gdzie backend aplikacji jest hostowany. W procesie wdrażania:

- Utworzono serwis dla aplikacji Node.js, integrując go z repozytorium GitHub.
- Skonfigurowano automatyczne wdrażanie kodu po każdej aktualizacji.
- Baza PostgreSQL została skonfigurowana poprzez zmienne środowiskowe (`DATABASE_URL`).
- Dostosowano backend do obsługi SSL i portu wymaganego przez Render.

Poniżej sie znajdują przykłady kodu do implementacji wybranych funkcji

1. **Połączenie z bazą danych config/database.js:**

```
const { Sequelize } = require('sequelize');
const config = require('./config');
const env = process.env.NODE_ENV || 'development';
const dbConfig = config[env];

const sequelize = new Sequelize(dbConfig.url, {
  dialect: dbConfig.dialect,
  logging: dbConfig.logging,
  dialectOptions: dbConfig.dialectOptions || {},
});

const connectDB = async () => {
  try {
    await sequelize.authenticate();
    console.log(`Connected to the Database (${env})`);
  } catch (error) {
    console.error(`Error with connection to the Database (${env})`, error)
  }
};

module.exports = { sequelize, connectDB };
```

Rysunek 11: Kod połączenia z bazą danych

2. **Obsługa tras API – routes/gyms.js:**

```
const express = require("express");
const router = express.Router();
const catchAsync = require("../utilities/catchAsync");
const gyms = require("../controllers/gyms");
const { isLoggedIn, isAuthor, validateGym } = require("../middleware");
const multer = require("multer");
const { storage } = require("../cloudinary");
const upload = multer({ storage });

router.route("/")
    .get(catchAsync(gyms.index))
    .post(isLoggedIn, upload.array("image"), validateGym, catchAsync(gyms.createGym));


router.get("/new", isLoggedIn, gyms.renderNewForm);

router.route("/:id")
    .get(catchAsync(gyms.showGym))
    .put(isLoggedIn, isAuthor, upload.array("image"), validateGym, catchAsync(gyms.updateGym))
    .delete(isLoggedIn, isAuthor, gyms.destroyGym);

router.get("/:id/edit", isLoggedIn, isAuthor, catchAsync(gyms.renderEditForm));

module.exports = router;
```

Rysunek 12: Kod obsługi tras

3. **Autoryzacja użytkowników (logowanie/rejestracja) – controllers/users.js:**

```
const User = require("../models/user");

module.exports.renderRegister = (req, res) => {
    return res.render("users/register");
};

module.exports.register = async (req, res, next) => {
    try {
        const { email, username, password } = req.body;
        // const hashedPassword = await bcrypt.hash(password, 10);
        const registerUser = await User.create({ email, username, password });
        req.login(registerUser, err => {
            if (err) {
                return next(err);
            }
            req.flash("success", "Welcome to FitnessHub!");
            return res.redirect("/gyms");
        })
    }
    catch (e) {
        req.flash("error", e.message);
        return res.redirect("/register");
    }
};

module.exports.renderLogin = (req, res) => {
    return res.render("users/login");
};

module.exports.login = (req, res) => {
    req.flash("success", "Welcome back!");
    const redirectUrl = res.locals.returnTo || "/gyms";
    return res.redirect(redirectUrl);
};

module.exports.logout = (req, res, next) => {
    req.logout(function (err) {
        if (err) {
            return next(err);
        }
        req.flash("success", "You've successfully logged out");
        return res.redirect("/gyms");
    });
};
```

Rysunek 13: Kod obsługi użytkowników

4.8.1 Implementacja wybranych funkcjonalności systemu

Zaimplementowane funkcjonalności obejmują:

- Logowanie i rejestrację użytkowników.
- Przeglądanie siłowni w formie listy i na mapie.

- Wystawianie recenzji oraz system oceniania.
- Zarządzanie sesjami użytkowników.

5 Testowanie systemu

W ramach rozwoju projektu przeprowadzono szereg testów mających na celu zweryfikowanie poprawności instalacji, konfiguracji oraz działania kluczowych funkcjonalności. Testy obejmowały sprawdzenie poprawności rejestracji i logowania użytkowników, dodawania oraz edycji siłowni, wystawiania recenzji, a także działania interaktywnej mapy lokalizacji siłowni. Dodatkowo zweryfikowano responsywność interfejsu oraz poprawność komunikatów systemowych informujących użytkownika o pomyślnym wykonaniu akcji. Poniżej przedstawiono kluczowe aspekty procesu testowania.

5.1 Instalacja i konfigurowanie systemu

Przed rozpoczęciem testowania funkcjonalności systemu WellFitness przeprowadzono proces instalacji i konfiguracji całego środowiska programistycznego. Wdrożenie obejmowało instalację i konfigurację serwera bazy danych PostgreSQL oraz narzędzia administracyjnego pgAdmin 4, które umożliwia zarządzanie bazą danych oraz przeprowadzanie operacji na tabelach. Jako środowisko programistyczne wykorzystano Visual Studio Code, które zapewnia wygodne narzędzia do implementacji zarówno backendu w Node.js, jak i frontendu bazującego na EJS, HTML, CSS i JavaScript.

System został wdrożony na platformie Render.com, gdzie utworzono dedykowany serwer dla aplikacji backendowej oraz bazę danych PostgreSQL. Integracja z repozytorium GitHub pozwoliła na automatyczne wdrażanie zmian kodu po każdej aktualizacji w głównym repozytorium.

Konfiguracja systemu była kluczowym i wymagającym etapem wdrożenia, który wymagał dokładnego dostosowania ustawień oraz testowania interakcji pomiędzy komponentami. Dzięki zastosowaniu nowoczesnych narzędzi, takich jak pgAdmin 4, Visual Studio Code i Render.com, udało się stworzyć stabilne i skalowalne środowisko pracy, umożliwiające szybkie wdrażanie oraz efektywne testowanie aplikacji.

5.2 Testowanie opracowanych funkcji systemu

W tym rozdziale sie znajduje opis testów jednostkowych, testów walidacji danych oraz testów manualnych.

5.2.1 Testowanie funkcji

Testy aplikacji FitnessHub zostały przeprowadzone w środowisku Node.js przy użyciu biblioteki `supertest` oraz frameworka `jest`. Ich celem było zweryfikowanie poprawności działania endpointów API, w szczególności:

- renderowania strony rejestracji,
- poprawności przekierowań po rejestracji i logowaniu,
- walidacji danych logowania,
- wylogowania użytkownika,
- listowania obiektów (siłownie).

Początkowo test dotyczący renderowania strony rejestracji zwracał kod statusu 200 zamiast oczekiwanego 300, co powodowało błąd:

[language=JavaScript] const response = await request(app) .get('/register') .expect(300);

Po poprawieniu kodu oczekiwany status został zmieniony na 200:
[language=JavaScript] const response = await request(app) .get('/register') .expect(200);

Po tej poprawce testy zostały ponownie uruchomione, a wszystkie przypadki zakończyły się sukcesem. Poniżej przedstawiono wyniki testów w terminalu:

```
59  describe('Render Register Page', () => {
60    test('should render the registration page', async () => {
61      const response = await request(app)
62        .get('/register')
63        .expect(300);
64
65      expect(response.text).toContain('Register');
66    });
67  });
68
```

```
OUTPUT   DEBUG CONSOLE   TEST RESULTS   PROBLEMS 1   TERMINAL   PORTS

    Synchronisation with the database successfully completed

    at log (app.js:27:17)

  ● Render Register Page › should render the registration page

    expected 300 "Multiple Choices", got 200 "OK"

      61 |     const response = await request(app)
      62 |       .get('/register')
    > 63 |       .expect(300);
         |        ^
      64 |
      65 |     expect(response.text).toContain('Register');
      66 |   });

      at Object.expect (testing/users.test.js:63:8)
      ----
      at Test._assertStatus (node_modules/supertest/lib/test.js:252:14)
      at node_modules/supertest/lib/test.js:308:13
      at Test._assertFunction (node_modules/supertest/lib/test.js:285:13)
      at Test.assert (node_modules/supertest/lib/test.js:164:23)
      at Server.localAssert (node_modules/supertest/lib/test.js:120:14)


  > Test run finished at 28.01.2025, 12:31:52 <

  Test Suites: 1 failed, 1 passed, 2 total
  Tests:       1 failed, 8 passed, 9 total
  Snapshots:   0 total
  Time:        3.372 s
  Ran all test suites related to changed files.
```

Rysunek 14: Testy funkcji (1)

```
PS D:\STUDIES\5 semestr\Bazy Danych\FitnessHub> npm test

> fitnesshub@1.0.0 test
> jest

 PASS  testing/gyms.test.js
  ● Console

    console.log
      Connected to the Database (test)

      at log (config/database.js:15:13)

    console.log
      Synchronisation with the database successfully completed

      at log (app.js:27:17)

 PASS  testing/users.test.js
  ● Console

    console.log
      Connected to the Database (test)

      at log (config/database.js:15:13)

    console.log
      Synchronisation with the database successfully completed

      at log (app.js:27:17)


Test Suites: 2 passed, 2 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        3.761 s, estimated 4 s
Ran all test suites.
```

Rysunek 15: Testy funkcji (2)

```
OUTPUT   DEBUG CONSOLE   TEST RESULTS   PROBLEMS   TERMINAL   PORTS

TestRun "FitnessHub:watch-tests-0:process-start:0 (5)" started
PASS testing/gyms.test.js
  ● Console

    console.log
      Connected to the Database (test)

      at log (config/database.js:15:13)

    console.log
      Synchronisation with the database successfully completed

      at log (app.js:27:17)

PASS testing/users.test.js
  ● Console

    console.log
      Connected to the Database (test)

      at log (config/database.js:15:13)

    console.log
      Synchronisation with the database successfully completed

      at log (app.js:27:17)

Test Suites: 2 passed, 2 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        3.527 s
Ran all test suites related to changed files.
```

```
✓ Test run at 1/28/2025, 12:37:34 PM
  ✓ GET /gyms should list all gyms and return status 200
  ✓ GET /gyms/new should render form if logged in
  ✓ should register a new user and redirect
  ✓ should render the registration page
  ✓ should render the login page
  ✓ should render the login page
  ✓ should log in a user and redirect to /gyms
  ✓ should fail to log in with invalid credentials and redirect to /login
  ✓ should log out a user and redirect to /gyms
✓ Test run at 1/28/2025, 12:37:01 PM
  ✓ GET /gyms should list all gyms and return status 200
  ✓ GET /gyms/new should render form if logged in
  ✓ should register a new user and redirect
  ✓ should render the registration page
  ✓ should log in a user and redirect to /gyms
  ✓ should fail to log in with invalid credentials and redirect to /login
  ✓ should log out a user and redirect to /gyms
  ✓ should render the login page
  ✓ should render the login page
```

Rysunek 16: Testy funkcji (3)

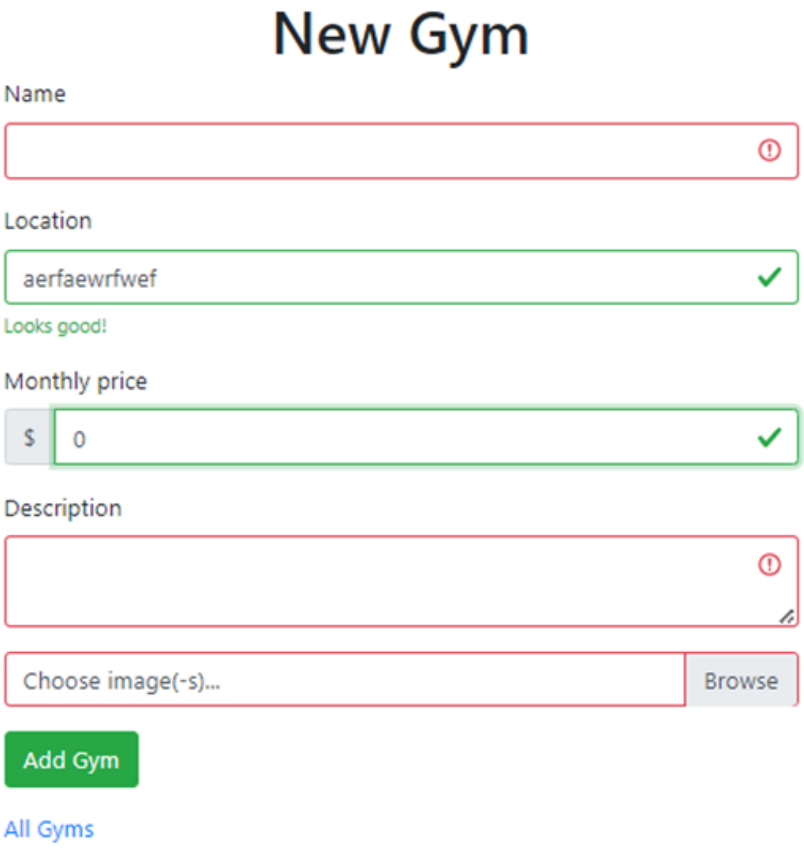
```
✓ 59  describe('Render Register Page', () => {
✓ 60    test('should render the registration page', async () => {
  61      const response = await request(app)
  62        .get('/register')
  63        .expect(200);
  64
  65      expect(response.text).toContain('Register');
  66    });
  67  });
  68
```

```
OUTPUT   DEBUG CONSOLE   TEST RESULTS   PROBLEMS   TERMINAL   PORTS

  ● Console

    console.log
      Connected to the Database (test)

      at log (config/database.js:15:13)

    console.log
      Synchronisation with the database successfully completed

      at log (app.js:27:17)

PASS testing/users.test.js
  ● Console

    console.log
      Connected to the Database (test)

      at log (config/database.js:15:13)

    console.log
      Synchronisation with the database successfully completed

      at log (app.js:27:17)

Test Suites: 2 passed, 2 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        3.524 s, estimated 4 s
Ran all test suites related to changed files.
```

Rysunek 17: Testy funkcji (4)

5.2.2 Testy walidacji danych

W ramach testowania systemu FitnessHub przeprowadzono testy walidacji danych w formularzu dodawania nowej siłowni. Sprawdzano poprawność wprowadzanych wartości oraz reakcję systemu na nieprawidłowe dane. Walidacja obejmowała wymagane pola, takie jak nazwa siłowni, lokalizacja, cena miesięczna oraz opis. Na zrzucie ekranu widoczne są błędy walidacji dla pól „Name” i „Description”, które zostały oznaczone na czerwono, co wskazuje na ich brak. Pole „Location” zostało wypełnione poprawnie, a system potwierdził jego prawidłowość zielonym znacznikiem. Podobnie zweryfikowano poprawność ceny miesięcznej, akceptując wartość „0”. System nie pozwala na dodanie siłowni bez wymaganych informacji, co zapobiega wprowadzaniu niekompletnych danych i zapewnia spójność przechowywanych informacji. Testy potwierdziły, że walidacja działa zgodnie z założeniami, poprawnie identyfikując brakujące wartości oraz przekazując użytkownikowi czytelne komunikaty o błędach.



Rysunek 18: Test walidacji danych

5.2.3 Testy manualne

W ramach testowania systemu FitnessHub przeprowadzono również testy manualne, których celem było zweryfikowanie poprawności działania kluczowych funkcji aplikacji. Testy obejmowały ręczne sprawdzenie procesu rejestracji i logowania użytkowników, dodawania i edycji siłowni, wystawiania recenzji oraz poprawności działania interaktywnej mapy

z lokalizacjami siłowni. Dodatkowo przetestowano walidację formularzy, sprawdzając, czy system poprawnie wykrywa brakujące pola i uniemożliwia dodanie niekompletnych danych. Weryfikacji poddano również mechanizmy autoryzacji, sprawdzając, czy niezalogowany użytkownik ma ograniczony dostęp do funkcji dodawania i edycji siłowni. Przeprowadzono testy na różnych urządzeniach i przeglądarkach, co potwierdziło responsywność oraz stabilność aplikacji. Wszystkie testy zakończyły się pomyślnie, a system działa zgodnie z założeniami, zapewniając poprawną obsługę użytkowników oraz danych. Jedyną zauważoną wadą systemu jest dłuższy czas ładowania stron, co wynika z konieczności pobierania dużych zbiorów danych oraz interakcji z bazą danych w czasie rzeczywistym.

5.3 Wnioski z testów

Na podstawie przeprowadzonych testów stwierdzono, że system działa poprawnie i zgodnie z założeniami projektowymi.

6 Podsumowanie

Projekt FitnessHub stanowi kompleksowe podejście do stworzenia funkcjonalnego systemu zarządzania siłowniami, opartego na bazie danych PostgreSQL. W trakcie realizacji systemu skupiono się na zapewnieniu przejrzystego interfejsu użytkownika, intuicyjnej obsługi oraz sprawnej komunikacji między klientem a serwerem. Szczególny nacisk położono na projektowanie i optymalizację bazy danych, co pozwoliło na efektywne przechowywanie i przetwarzanie informacji dotyczących użytkowników, siłowni, recenzji oraz lokalizacji. Zastosowany model komunikacji klient-serwer został oparty na architekturze trójwarstwowej, w której backend napisany w Node.js i Express.js odpowiada za obsługę operacji biznesowych, natomiast frontend umożliwia interakcję użytkownika z systemem poprzez przeglądarkę internetową. Aplikacja została wdrożona na Render.com, co zapewniło jej stabilne działanie i dostępność online. Podczas implementacji kluczową rolę odegrały mechanizmy walidacji danych, które zabezpieczają system przed wprowadzaniem błędnych informacji, zarówno na poziomie formularzy użytkownika, jak i operacji bazodanowych. Wykorzystano również interaktywne mapy do wizualizacji lokalizacji siłowni, co znacząco podnosi użyteczność systemu. W ramach testów sprawdzono poprawność działania głównych funkcji, takich jak rejestracja i logowanie użytkowników, dodawanie i edytowanie siłowni oraz wystawianie recenzji. Przeprowadzone testy jednostkowe oraz manualne potwierdziły poprawne funkcjonowanie systemu, umożliwiając jednocześnie identyfikację i naprawę drobnych błędów. Jedyną zauważoną wadą jest dłuższy czas ładowania stron, wynikający z konieczności pobierania dużej ilości danych oraz komunikacji z serwerem. Projekt FitnessHub spełnia założone wymagania i stanowi solidną podstawę do dalszego rozwoju oraz rozbudowy o dodatkowe funkcjonalności, takie jak bardziej rozbudowany system zarządzania siłowniami czy dodatkowe opcje personalizacji dla użytkowników.

7 Literatura i źródła

1. **Dokumentacja PostgreSQL** – PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database.* Dostępne online: `https://www.postgresql.org/docs/`

2. **Node.js Documentation** – OpenJS Foundation. *Node.js API Documentation.* Dostępne online: `https://nodejs.org/en/docs`

3. **Express.js Documentation** – *Express Web Framework.* Dostępne online: `https://expressjs.com/`

4. **Render.com Documentation** – Render Platform, *Hosting and Deployment for Web Applications.* Dostępne online: `https://render.com/docs`

5. **pgAdmin Documentation** – *PostgreSQL Management Tool.* Dostępne online: `https://www.pgadmin.org/docs/`

6. **JWT Authentication** – *JSON Web Tokens for Secure Authentication.* Dostępne online: `https://jwt.io/introduction/`

7. **REST API Design** – Richardson, L. *RESTful Web APIs: Services for a Changing World.* O'Reilly Media, 2013.

8. **Frontend Technologies** – Mozilla Developer Network (MDN). *HTML, CSS, and JavaScript Documentation.* Dostępne online: `https://developer.mozilla.org/`

9. **Bootstrap Framework** – *Responsive Frontend Development.* Dostępne online: `https://getbootstrap.com/`

10. **Systemy baz danych** – Elmasri, R., Navathe, S.B. *Fundamentals of Database Systems.* Pearson Education, 2015.